

UAS E2EE Data Protection Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS E2EE Data Protection Implementation Guide.....	3
2. Introduction to UAS E2EE Data Protection.....	4
3. UAS E2EE Data Protection Configuration.....	15
4. UAS E2EE Data Protection API Use Cases and Sample Codes.....	19
5. Appendices.....	30

1. Preface - UAS E2EE Data Protection Implementation Guide

The AccessMatrix Universal Authentication (UAS) Server provides End-to-End Encryption (E2EE) Data Protection (DP) capabilities. This document provides information on UAS E2EE DP, as well as important implementation and integration details.

Intended Reader

It is intended as a reference for the security administrator, professional service and developers.

Document Scope

This document covers the concept, implementation steps, and API integration details for AccessMatrix Universal Authentication Server (UAS) End-to-End Encryption (E2EE) Data Protection (DP).

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents:

- [*UAS Administration Guide*](#)
- [*UAS E2EE Data Protection Lite Application Developer Guide*](#)
- [*AccessMatrix REST API Specification*](#)

2. Introduction to UAS E2EE Data Protection

E2EE Data Protection (E2EE DP)

Transaction data are typically transmitted from the client (browser) to an application using Transport Layer Security (TLS) protection, but most of the cases they are not end-to-end. TLS encryption ends typically at the load balancer or web server or point-to-point. Sensitive information may get exposed in the internal network between the web server and application server.

E2EE Data Protection (E2EE DP) works by encrypting the transaction data at the user's browser or mobile app before the data is TLS-encrypted. The encrypted data remains encrypted when TLS is decrypted at the web server or any intermediary point. E2EE DP uses a combination of public-private key cryptography, symmetric key encryption, and additional of one-way hashing function.

Components of E2EE Data Protection

- **E2EE DP client-side library**

It is called by client-side web/mobile application to encrypt data in the browser or mobile app.

- **E2EE DP application-side library (E2EE DP SDK)**

It is called by the server-side web application to decrypt the encrypted data.

The E2EE DP SDK uses a ConfigProvider interface, which abstracts connectivity to the AccessMatrix server to retrieve the Data Protection Client Configuration. It uses a pluggable implementation, so different ConfigProviders can be used (e.g. REST ConfigProvider or XML-RPC ConfigProvider). An official REST ConfigProvider is bundled with AccessMatrix and is the default E2EE DP implementation.



Important Note: For documentation on the older version of the E2EE DP SDK which uses XML-RPC libraries, refer to *UAS E2EE Data Protection Implementation Guide 5.7.2-GA*.

- **AccessMatrix API**

It is used to communicate to the AccessMatrix server.

- **AccessMatrix UAS**

It is a policy server that handles key management.

Features of E2EE Data Protection

- Confidentiality (end-to-end encryption of data)
 - Integrity check
-

-
- Replay attack protection
 - Key management/storage (optionally, keys in storage can be protected by HSM key)
 - Bidirectional (only for Java/Android and iOS clients)
 - Bidirectional encryption is used when there is a need to encrypt data for server-to-client communication. For example, the application server needs to send an account number to the mobile app.
 - Data Protection Policy
 - Authorization to specify which application/agent can use the Data Protection keys

E2EE DP Lite Application and E2EE DP Full Application

There are two versions of E2EE DP:

- **E2EE DP Lite (standalone) Application:** This version can be run without AccessMatrix server.
- **E2EE DP Full Application:** This version requires AccessMatrix server.

The comparison table below lists the differences between Lite and Full versions.

Feature	Lite Version	Full Version
Key storage management	Application (server-side)	AccessMatrix server
Key rollover	Not supported	Supported
Key wrapping using HSM	Not supported	Supported
Number of max key pairs	4	16
Unidirectional (client to server)	Supported	Supported
Bidirectional (server to client)	Not supported	Supported (Android/iOS Client SDK)
Symmetric encryption (for long data)	Supported	Supported
Client libraries	Javascript, Java/Android, iOS	Javascript, Java/Android, iOS

Feature	Lite Version	Full Version
Application (server-side) libraries	Java, .NET, C Native, PHP	Java
Application (server-side) configuration	Manual (application-managed)	Automatically pulled from AccessMatrix server
Endpoint support	Application (server-side)	Application (server-side), AccessMatrix server, Application (decryption by AccessMatrix server)

For more information on the Lite-version, refer to the [UAS E2EE Data Protection Lite Application Developer Guide](#).

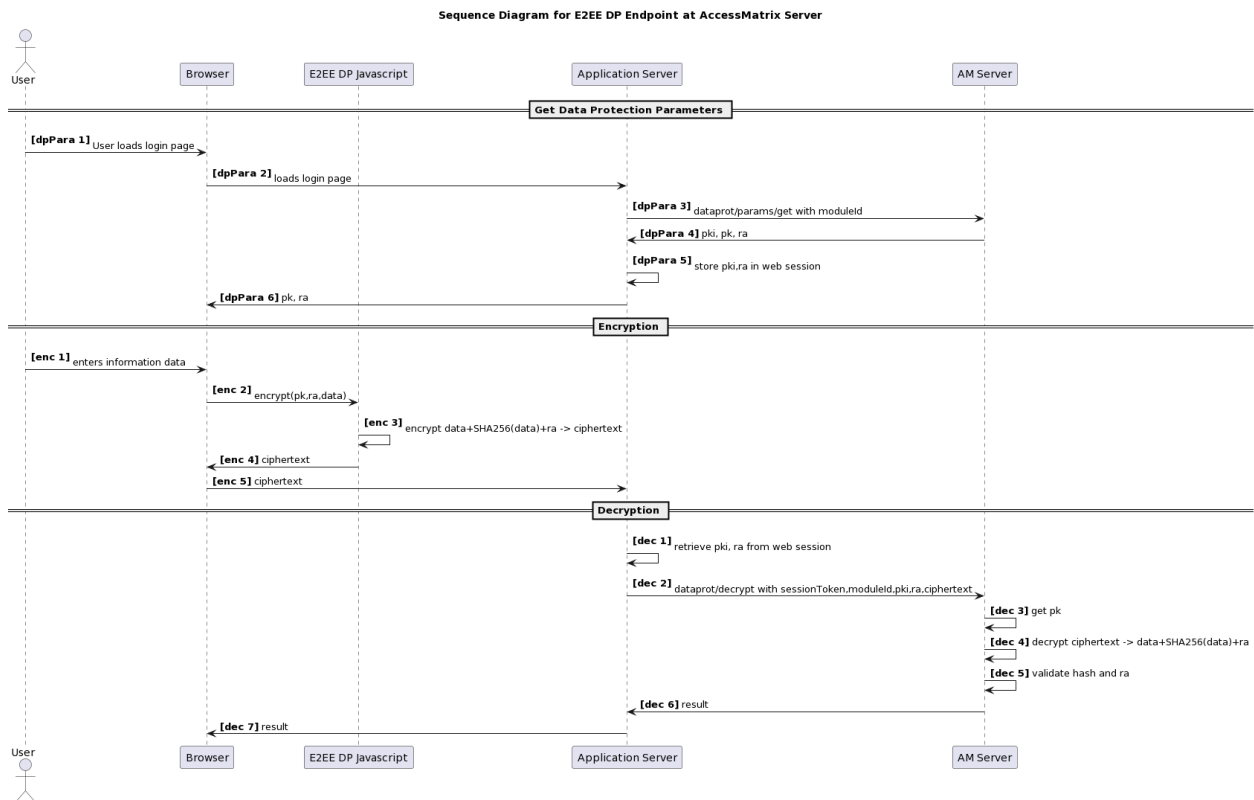
E2EE DP Endpoints

E2EE DP supports different decryption endpoints. The endpoint is where the data is decrypted and who receives the decrypted data/plain text. The endpoint needs to be configured according to the use case/scenario.

Endpoint at AccessMatrix UAS

AccessMatrix UAS does the decryption and plain text is only exposed to it. This option protects AccessMatrix authentication credential by providing another layer of authentication credential protection (for example to encrypt non-E2EEA password and E2EE OTP protection use case). It works with AccessMatrix authentication-related API: login, reauthenticate, verifyPassword, changePassword, resetPassword, verifyOtp, verifyDigitalSignature, syncTokenAndHostBySerialNum. E2EE DP SDK/Application server-side library is not required.

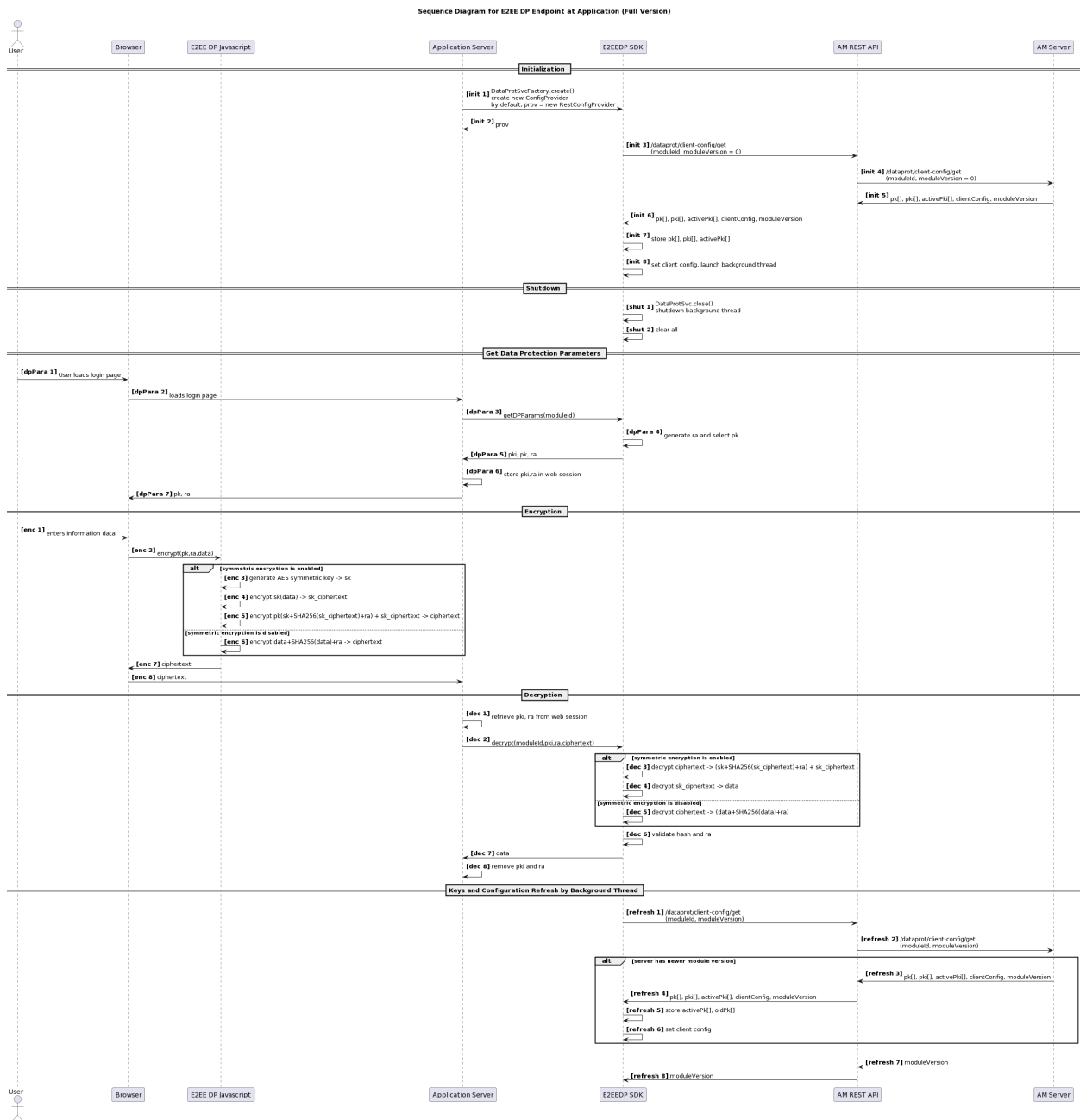
Sequence Diagram for E2EE DP Endpoint at AccessMatrix Server



Endpoint at Application Server (E2EE DP SDK)

Decryption is done by the E2EE DP SDK running on the Application Server. Private keys are self-generated (Lite version) by E2EE DP SDK or retrieved (full version) from AccessMatrix server. This option is suitable for protection of Application's transaction data. Decryption is done inside the E2EE DP SDK running on the Application Server and does not go through AccessMatrix Server. Hence the performance is faster from the lesser network hop. E2EE DP SDK/Application server-side library is required.

Sequence Diagram for E2EE DP Endpoint at Application (Full Version)



Bidirectional Data Protection

In most use cases, the data protection required is only uni-directional. It refers to only the client needs to encrypt sensitive data before submitting to the server. Occasionally, the server will also be required to encrypt sensitive data to send to the client. Thus, the data protection becomes bi-directional.

Features

- Endpoint at the application server

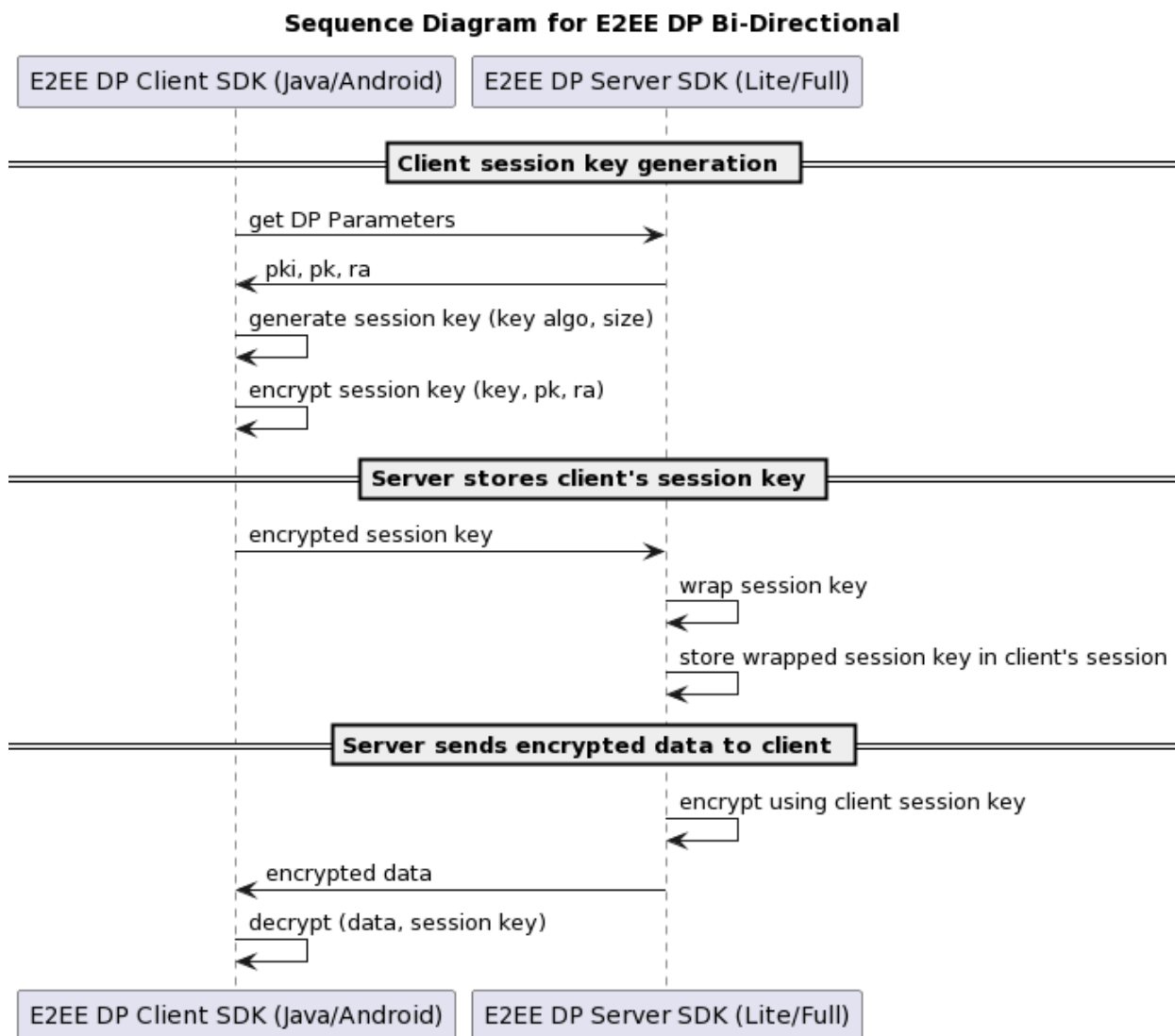
- **E2EE DP Client SDK**

- Only supported for Java/Android and iOS
- Additional functions:
 - generate session (symmetric) key
 - encrypt session key before sending to server

- **E2EE DP SDK (Server)**

- Both Lite and Full versions are supported
- Additional functions:
 - The wrap session key (decrypt session key from client and re-encrypt for storing in the web session)
 - Encrypt data using client session key before sending to the client

Sequence diagram



Implementation Checklist

Below is the checklist for implementing UAS E2EE DP solution.

Tasks	Reference
Create the Wrapping Key	Wrapping Key
Create the Data Protection Module	Data Protection Module

Implementation Planning

Planning prior to environment setup is critical to the success of UAS E2EE DP solution. Understanding the customer's nature of business, studying the customer's requirements and its existing environments will help to identify the actions to make prior to actual implementation.

	AccessMatrix Server Endpoint	Application Server Endpoint
Sensitive application private data	Is exposed on AccessMatrix server	Not exposed to AccessMatrix server
Performance	Affected - slow	Not affected - fast
E2EE DP SDK required	Not required	Required (refer to Integration Consideration - Application Server Endpoint for more information) • Java only • Requires connection to AccessMatrix server
App server platform does not support E2EE DP SDK	Not supported • Application calls /rest/dataprot REST service for all DP functions	Supported
HSM	Optional • Configure to use HSM wrapping key in DP module	Optional • Configure to use HSM wrapping key in DP module

The following sections discusses on the implementation considerations that should be assessed during planning.

Security Consideration

AccessMatrix Server Endpoint

- Less secure
- Decryption on AccessMatrix server
- Sensitive data is exposed on AccessMatrix server

Application Server Endpoint

-
- More secure
 - Decryption on the application server
 - Sensitive data is not exposed to AccessMatrix server

Integration Consideration

AccessMatrix Server Endpoint

- E2EE DP SDK is not required
- Application server only need to call REST service

Application Server Endpoint

- **Full version**
 - Requires a connection to AccessMatrix server
 - Requires E2EE DP SDK Full version (supports only Java)
- **Lite version**
 - Does not require a connection to AccessMatrix server
 - Requires E2EE DP SDK Lite version (supports Java, C, .NET, PHP)

E2EE DP Client SDK

For uni-directional data protection (client to server):

- **Browser client**
 - Javascript SDK
- **Mobile app**
 - Android (Java) SDK
 - iOS SDK
- **Java-based client**
 - Java SDK
- **Other clients (non-Java, non-Javascript)**
 - C SDK
 - client must be able to interface to C library

For bi-directional (server-to-client as well):

-
- Java/Android SDK
 - iOS SDK

Performance Consideration

AccessMatrix Server Endpoint

- Decryption performed by AccessMatrix server machine.
- Slower performance due to:
 - Increased and shared processor load.
 - Network latency between the app and AccessMatrix server.
- May be improved if HSM is used to perform decryption.

Application Server Endpoint

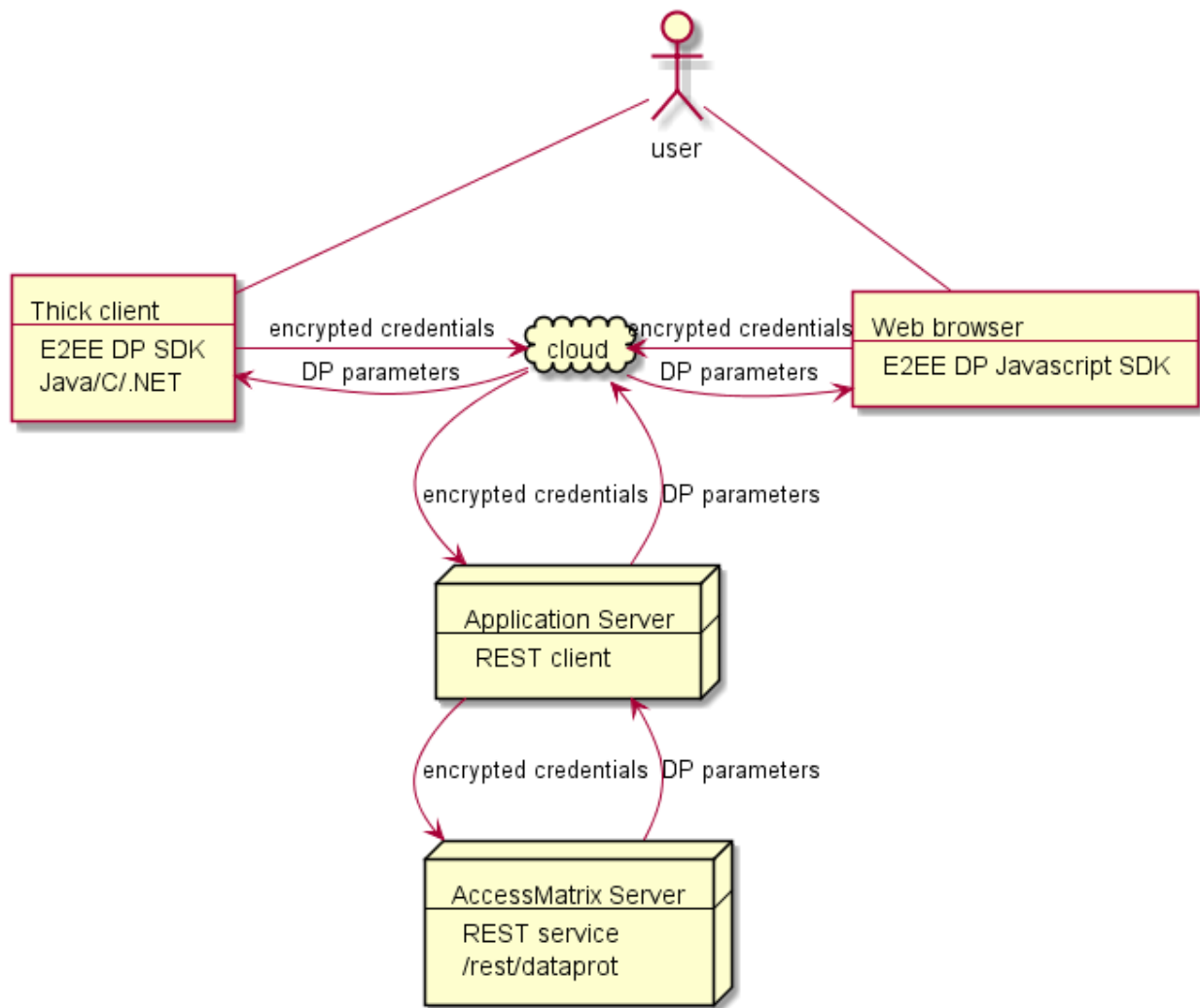
- Decryption performed by an application server machine.
- Better performance due to:
 - No increased and shared processor load for AccessMatrix machine.
 - Increased and shared processor load for the application server machine.
 - No network latency between AccessMatrix and app server.

E2EE Data Protection Deployment

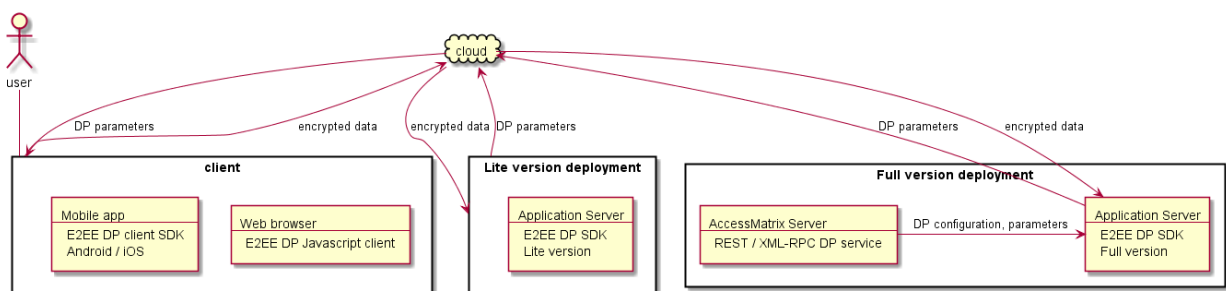
AccessMatrix Server Endpoint

Applicable scenarios include:

- Using E2EE DP only to secure credentials when calling various APIs that accept password/credential parameters on the AccessMatrix server.
 - Application server platform cannot use E2EE DP SDK at all; it calls the REST API for all DP functions.
-



Application Server Endpoint



3. UAS E2EE Data Protection Configuration

This section defines the steps relating to the implementation of UAS E2EE DP. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of UAS E2EE DP solution.

Regardless of whether the data protection endpoint terminates at AccessMatrix server or application server, the basic setup and configuration steps are similar.

- **Upload E2EEA License**

Server license is used to enable general and product specific features. For E2EE DP license, refer to [Upload E2EE DP License](#) for details.

- **Create Wrapping Key**

Key wrapping is the process of encrypting one key using another key to store it or transmit it over an untrusted channel securely. To support this key wrapping process, a wrapper key should be created. This key uses symmetric encryption to encapsulate key material. Refer to [Create Wrapping Key](#) for the configuration details.

- **Create Data Protection Module**

Data protection module allows the administrator to configure the endpoint of E2EE DP, the asymmetric and symmetric ciphers, and the algorithm for integrity checking. Refer to [Create Data Protection Module](#) for the configuration details.

Upload E2EE DP License

Before doing any configuration in the Admin Console, secure the server license required to implement UAS E2EE Data Protection. Ensure that the **E2EE Data Protection** license feature is set to "Yes" to enable E2EE DP support.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
 2. Select the Server Id and click **Edit**.
 3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
 4. Click **Upload License File** to upload the license file to the server.
 5. Save the changes.
-

-
- Restart the AccessMatrix server for the new license to take effect.

Create Wrapping Key

- Log in to Admin Console using administrative right.
- Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key**.
- Click **Create**.
- Select "SystemKeyProvider" for the key provider.
- Select "Symmetric key (secret key)" for the **Encryption key template** and click **Next**.
- Enter an encryption key Id, the encryption key name and its description.
- Select "Key Wrapping" for the **Purpose** attribute.
- Select the appropriate **JCE Provider**.
- Select the other attribute values according to the required security policy.



Note: For AES keys above 128-bit, update JRE to Unlimited JCE (Java Cryptography Extension) Policy.

- Save the changes.

Create Data Protection Module

- Navigate to **Administration Dashboard > Configuration > Encryption > Data Protection**.
 - Click **Create**.
 - Select "Unidirectional Data Protection" for the implementation.
 - Click **Next**.
 - Enter a ***Data Protection Module Id**, a ***Data Protection Module Name** and its **Description**.
 - Select one of the following for **Endpoint**:
 - Select "Application Server (E2EE DP SDK)" to use the application server. It specifies that the encrypted data will be decrypted by the application and not by the AccessMatrix server.
 - Select "Application Server via AM5 Server" to use the application server via the AccessMatrix Server. It specifies that the encrypted data will be decrypted by the application via the AccessMatrix server.
-

-
- Select "AM5 Server" to use the AccessMatrix server. It specifies that the encrypted data will be decrypted at the AccessMatrix Server and the cleartext will be made available to the AccessMatrix Server only.
7. Specify the number of RSA key-pairs available using the **Number of Key Pairs** attribute.
 8. Select the key size using the **Key size** attribute.

i **Note:** RSA 2048-bit key can directly encrypt a 166-character (bytes) cleartext. RSA 1024-bit key can directly encrypt a 38-character (bytes) cleartext.

9. Select the wrapping key using the **Wrapping Key** attribute.

i **Note:** The wrapping keys will wrap the asymmetric keys generated by this module. For more details on how to create the wrapping key, refer to [Create Wrapping Key](#).

10. (Optional) Set the **Enable PQC** attribute to "true" if Post-Quantum Cryptography (PQC) encryption is required.
11. Specify the desired encryption algorithm using the **PQC Algorithm** attribute, if **Enable PQC** attribute has been set to "true".

i **Note:** By default, the algorithm selected is "ML-KEM-768". The number indicates the key size; a larger key size provides stronger encapsulation at the cost of performance.

12. Set the **Use symmetric key** attribute to "false".

i **Notes:** This attribute is used for the password. It should only be set to "true" if the cleartext to be encrypted is too long.

- For application server endpoint, this is typically true since it is expected that data of various lengths would need to be encrypted using a symmetric key (otherwise using the asymmetric key will limit the length of data).
- Under **Algorithm**, the Symmetric Cipher Algorithm ("SM4") option is not applicable if the **Use symmetric key** attribute is "false".

13. Set the **Check for message integrity** attribute to "false".

i **Note:** This attribute must be set to "false" if **Use symmetric key** is "false". When a symmetric key is not being used, the AccessMatrix server will use RSA with OAEP/SHA-1 padding. As such, no other message hash should be specified. For an application server endpoint, this attribute should typically be set to "true".

14. Set the **Algorithm** attribute under the **Integrity Check** section to "SHA-256".
 15. Click the **Save** button to save the changes.
-



Important Note: Once the data protection module has been successfully created, click on the **Generate Key Pairs** button. If PQC has been enabled, the user may select the algorithm for the keys to be generated. Upon completion, the Generate Key Pairs dialog box will display the list of generated keys. You may click on the **View Keys Status** button to view the list of generated keys and their status within this data protection module.

The following attributes can be left as default or according to the security policy.

- **Expiry Time of Old Keys (in minutes)**

When new keys are generated, previously generated keys are still valid for the configured time.

- **Random number length**

This attribute indicates the length of the random number being generated.

- **API's cache refresh interval (in minutes)**

4. UAS E2EE Data Protection API Use Cases and Sample Codes

The following sections describe the APIs that can be used for each use case scenario.

Get Data Protection Parameters

All data protection use cases, regardless of whether lite or full version, endpoint at AccessMatrix server or application, requires the client to obtain data protection parameters before it can perform any encryption.

REST API
http(s)://am-host:am-port/rest/dataprot/params/get
Request sample
JSON <pre>{ "sessionToken": "...admin session with View System rights", "moduleId": "DP002", "params": { "PQC": "preferred" } }</pre> Note: • sessionToken: The session token of an admin user, also called an admin session. An admin session is one of various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs. Other methods include: ○ Dynamic agent session ○ Access token Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs • moduleId: Id of your data protection module. • PQC (optional). ○ If specified as "required", PQC key-pairs will be returned. If there are no PQC key-pairs available, an error will be returned instead. ○ If specified as "preferred", PQC key-pairs will be returned if they are available. Otherwise, non-PQC key-pairs such as RSA key-pairs will be returned instead.
Response sample

REST API

JSON

```
{
  "pki": 4,
  "pk": "AB4F9D2B3AF7703F6BF0334C07F01F9F08B34DDAD45C636ED7A0BD97DE80E0FEAFF80CCB014A070363DAA248634BF8163B3689A5D4BC5D429EB1546329D8A66803977F161E89ED6B1B567A149EFBD242850FBA073DF205227D8F7372BA21D4940A15B1FDA11CDB5252FC3AF0D412513D62152A851D8B935D35B1A6E0068626E3D7247344AD49F99BFF56F2A0BC81ACEB39C5890D476AD83BD3EFC4E2CBF24619D0F4A60B44D6F2C6F0E7A806F9623799A9448E292D7513D689A93ECA66AB8FD44077806EDB42E8E9752A31B80023DED75E81DA5E8920F89EF45FC0E3A1F0CF3F7603D2A4635D7B275CE017808080938F25DFA5A625E06EA856A3E03A570AFA71,10001",
  "randomNumber": "A888EF672809EE204A2EB092BF97A78C",
  "e2eeSesToken": "0002v4cQw7MJNTv9XCqAdK6OdIk7Xv-Lx1llQkw7si7XAhRaZUax3sDJLYRNciSpNiMn55EIZ0OS0r8rRWIzQBs3qUx92w"
}
```

Notes:

- pki: public key index
- pk: public key
- randomNumber: server random number
- e2eeSesToken: end to end session token

Endpoint at AccessMatrix Server

This use case is where the data protection endpoint terminates at AccessMatrix server. It is used for encrypting the clear credential like password, OTP and others when making API calls like Login, Change Password, Verify OTP and more.

Login

1. Client gets data protection parameters as described in a previous section.
2. Assuming the login page is loaded in a web browser.
 - a. Login page to include the following Javascript files from the E2EE DP Client SDK:

```
<script language="javascript"src="amdp.js"></script>
<script language="javascript"src="json2.js"></script>
```

- b. After user enters password, the page shall call the following Javascript function to encrypt the password:

```
/*
 * pk, randomNumber, e2eeSesToken are from the data protection parameters
 * cipherParams is a JSON string
 ** it can be null if data protection module is not using symmetric key and integrity
check
 ** if data protection module is using symmetric key and integrity check, then the
same parameters must be set accordingly:
    cipherParams = "{\"hash\": true, \"hashAlgo\": \"SHA-256\", \"symmetric\":
true, \"symmetricAlgo\": \"AES\", \"symmetricKeyLength\": 128}"
 ** if data protection module has PQC enabled, then the PQC algorithm must be set
accordingly:
    cipherParams = '{"pqcAlgo":"PQC-0-768","hash":true,"hashAlgo":"SHA-256"}'
 */
amdp.encrypt(cipherParams, pk, randomNumber, password, e2eeSesToken);
```

3. When calling the REST login URL, extra parameters pertaining to data protection needs to be included in "params":
- DPMODULEId: module Id
 - Pki: public key index
 - Ra: random number

REST API
http(s)://am-host:am-port/rest/authn/login
Sample request
JSON <pre>{ "realmId": "SystemRealm", "userId": "user001", "password": "...the encrypted password...", "params": { "DPMODULEId": "DP002", "Pki": 4, "Ra": "A888EF672809EE204A2EB092BF97A78C" } }</pre>

Other Credential Verification: Verify Password, Reset Password, Verify OTP, Verify Response, Verify Digital Signature

For all other similar APIs that verify a single credential, apply the same steps as for login.

1. Get data protection parameters.
2. Include the E2EE DP Client SDK Javascript:
 - a. Encrypt the credential entered by the user.
 - b. cipherParams must match those configured in the data protection module.
3. When calling the Verification API:
 - a. Set the encrypted credential instead of the clear one.
 - b. Set the other "params" for data protection:
 - DPMODULEID: module ID
 - PKI: public key index
 - Ra: random number

Credential Change: Change Password APIs

For the Change Credential, Change Password APIs that require two credentials, the steps are similar except that two sets of data protection parameters are required to encrypt each credential separately.

1. Get two sets of data protection parameters.
 2. Include the E2EE DP Client SDK Javascript:
 - a. Encrypt the two credentials entered by the user with separate sets of data protection parameters.
 - b. cipherParams must match those configured in the data protection module.
 3. When calling the API:
 - a. Set the encrypted credentials instead of the clear ones.
 - b. Set the other "params" for data protection:
 - DPMODULEID: module ID
 - PKI1: public-key index (for the first credential)
 - Ra1: random number (for the first credential)
-

-
- Pki2: public-key index (for the second credential)
 - Ra2: random number (for the second credential)

Endpoint at Application Server

This use case is where the data protection endpoint terminates at the Application server. It is used for securing and protecting sensitive data between the application user and the application server. The application will use the Java-based E2EE DP SDK.

The E2EE DP SDK has two versions:

- Lite version - Application will initialize the lite DP SDK without any need to connect to the AccessMatrix server.
- Full version - Application will only need to connect to the AccessMatrix server to obtain the configured Data Protection Modules and Parameters.
- After that, encryption and decryption are performed in the application server without the involvement of AccessMatrix server.

Additionally, the default ConfigProvider for the Java-based E2EE DP SDK is based on the REST standard. It is pluggable, such that a different implementation (e.g. XML-RPC) can be injected instead if needed.

Application Server E2EE DP Life Cycle

The application server will typically be a WEB application or with WEB service backend. The E2EE DP SDK will be used in the application life cycle as follows:

1. Application Initialization/Startup

- a. Initialize E2EE DP SDK.
- b. A DataProtSvc service object will be created.
- c. The DataProtSvc object is threadsafe and meant to be used by concurrent threads across the application.
- d. A global reference to the DataProtSvc service object will be cached and made accessible to the entire application.
 - i. For a Java web application, this reference is typically set as a global application attribute.

2. Runtime Usage involves three steps

- a. The application gets DP parameters and presents them to the application client.
- b. Application client could be a web browser, mobile app or desktop app.
 - i. The client uses E2EE DP Client SDK to encrypt sensitive data with given DP parameters before passing back to the application server.
- c. The application server uses E2EE DP SDK decrypt method to recover the sensitive data from the client.

3. Application Shutdown

- a. Shut down or close E2EE DP SDK.

Application Server E2EE DP SDK Startup

The E2EE DP SDK is Java-based, and application is typically a Java web app. Application startup logic is commonly run in the `init()` method of an application servlet.

The following code sample is for the default REST ConfigProvider implementation:

Java

```
public void init(ServletConfig config) throws
ServletException {
    String amEndpoints = "http://192.168.50.103:8080";
    ConfigProvider prov = new
RestConfigProvider(amEndpoints);

    // full: provide the module IDs and other necessary
parameters
    String[] moduleIds = {"DP01", "DP02"};
    Map<String, String> params = new HashMap<String,
String>();
    // valid values for "PQC" are "preferred" (default)
or "required"
    params.put("PQC", "required");
    DataProtSvc dp = DataProtSvcFactory.create(prov,
moduleIds, params);

    // simple-lite: specify the number of key-pairs and
the algorithm
    // to use (ML_KEM_512, ML_KEM_768, or ML_KEM_1024)
    DataProtSvc dp =
DataProtSvcFactory.createSimpleLiteInstance(4,
"ML_KEM_768");

    // set dp as application attribute
```

```
config.getServletContext().setAttribute("DataProtection",  
dp);
```

Application Server E2EE DP SDK Get Params

Whenever the application requires a user to encrypt sensitive data for submission, it shall get DP parameters and dispatch them down to the client.

The following code sample is for the default REST ConfigProvider implementation:

Java

```
try {  
    DPParams dpParams = dp.getDPPParams(dpModuleId);  
    String dpRandomNo = dpParams.getRandomNumber();  
    String dpPublicKey = dpParams.getPk();  
    int dpPublicKeyIndex = dpParams.getPki();  
    String e2eeSesToken = dpParams.getE2eeSesToken();  
} finally {  
    // insert shutdown logic here  
}
```

Application Client E2EE DP Encryption

Client gets data protection parameters as described in the previous section. Assuming the client web page is loaded in a web browser, then the page will have to include the following Javascript files from the E2EE DP Client SDK.

HTML

```
<script language="javascript" src="amdp.js"></script>  
<script language="javascript"  
src="json2.js"></script>
```

After user enters the data to be protected, the page shall call the following Javascript function to encrypt the data before submitting to application server.

JavaScript

```
// RSA cipher params  
var cipherParams = '{"hash": true, "hashAlgo": "SHA-
```

```
256", "symmetric": true, "symmetricAlgo":
"AES", "symmetricKeyLength": 128}';

// PQC cipher params
var cipherParams = '{"pqcAlgo": "PQC-0-
768", "hash": true, "hashAlgo": "SHA-256"}';

var cipherText = amdp.encrypt(cipherParams, pk,
randomNumber, password, e2eeSesToken);
```

where

- pk (public key), randomNumber, e2eeSesToken are from the data protection parameters.
- cipherParams is a JSON string.
 - It can be null if the data protection module is not using symmetric key and integrity check.
 - More typically, if the data protection module is using symmetric key and integrity check, then the same parameters must be set accordingly.

Application Server E2EE DP SDK Decryption

Upon receiving the encrypted data from the client, the application server will call the `decrypt()` method to obtain clear data.

Java

```
/**
 * for decryption of ciphertext in string format
 */
PlainText plainText = dp.decrypt(moduleId, ciphertext,
publicKeyIndex, randomNumber);
String clearData = plainText.getAsString();

/**
 * for decryption of encrypted stream
 */
InputStream encryptedInputStream = ...;

// the HMAC of this input stream is required for
integrity checking
String hmac = ...;

// decrypt() writes the deciphered bytes to os1, but it
also returns OutputStream
// to be handled at the discretion of the caller; os1
```

```
and os2 need not be distinct
// (i.e they can be the same)
OutputStream os1 = ...;
OutputStream os2 = dp.decrypt(
    moduleId, encryptedInputStream, publicKeyIndex,
    randomNumber, hmac, os1);
```

- Application may need to remember the public key index and random number sent to the client if the client does not submit these back to the application.
- Typically, these can be stored as web session attributes before dispatching them down to the client.

Bi-Directional E2EE DP

Client Generates Session Key

The following example assumes the client is a Java/Android application.

The client gets data protection parameters as described in the previous section.

Java

```
import com.isprint.am.dp.api.client.DataProtClient;

// RSA cipher params
// you can use either the predefined params or a custom JSON string
// note: bidirectional DP requires integrity checking
String cipherParams =
DataProtClient.CIPHERPARAMS_SYMMETRIC_AES_128_INTEGRITY_SHA_256;
String cipherParams = "{\"hash\": true, \"hashAlgo\": \"SHA-256\",
    \"symmetric\": true, \"symmetricAlgo\": \"AES\", \"symmetricKeyLength\":
    128}";

// PQC cipher params
// you can use either the predefined params (default to ML-KEM-768) or a
// custom JSON string
// note: bidirectional DP requires integrity checking
String cipherParams =
DataProtClient.CIPHERPARAMS_PQC_MLKEM_768_INTEGRITY_SHA_256;
String cipherParams = "{\"pqcAlgo\": \"PQC-0-
    768\", \"hash\": true, \"hashAlgo\": \"SHA-256\"}";

byte[] sessionKey =
DataProtClient.generateSessionKey(DataProtClient.SESSION_KEY_SIZE_AES_128
);
String encryptedSessionKey =
DataProtClient.encryptSessionKey(cipherParams, publicKey, randomNumber,
    sessionKey);
```

where

-
- `publicKey`, `randomNumber`, `e2eeSesToken` are from the data protection parameters.
 - `cipherParams` is a JSON string corresponding to a data protection module using symmetric key and integrity check; the same parameters must be set accordingly.

The client should then store the generated session key locally in its session and submit the encrypted session key to the server.

Server Stores Wrapped Client Session Key

The server then wraps the encrypted client session key and store it in the client's web session (assuming the server is a web app). The server after that can use the client's session key to encrypt data to send to the client.

Java

```
String wrappedSessionKey = dp.wrapSessionKey(proxy,
moduleId,
                                encryptedSessionKey, pki,
randomNumber);
request.getSession().setAttribute("sessionKey",
wrappedSessionKey);

// encrypt data to send to client
CipherText cipherText =
dp.encryptUsingSessionKey(proxy, moduleId,
wrappedSessionKey, plaintext);
String ciphertext = cipherText.getAsHexBinString();
// string sent to client
```

Client Decrypts Data with Session Key

The client receives encrypted data from the server and decrypts using the session key.

Java

```
import com.isprint.am.dp.api.client.DataProtClient;
byte[] data = DataProtClient.decrypt(cipherParams,
sessionKey, ciphertext);
```

Application Server E2EE DP SDK Shutdown

Application shutdown logic is commonly run in the `destroy()` method of an application servlet.

Java

```
public void destroy() {  
    DataProtSvc dp = (DataProtSvc)  
    getServletContext().getAttribute("DataProtection");  
    if (dp!=null) {  
  
    getServletContext().removeAttribute("DataProtection");  
        dp.close();  
    }  
}
```

5. Appendices

A. Errors

A.1 DataProtectionException

For the Java E2EE DP SDK, the DataProvSvc (com.isprint.am.dp.api.DataProtSvc) methods will throw subclasses of DataProtectionException (com.isprint.am.dp.api.DataProtectionException).

1. DataProtectionException.RandomNumberMismatch - random number not matching
2. DataProtectionException.KeyNotFound - encryption/decryption key not found
3. DataProtectionException.KeyExpired - encryption/decryption key expired
4. DataProtectionException.MessageIntegrityCheckFailure - message integrity check failed

A.2 Javascript Client amdp Exception

The Javascript client amdp.js functions will throw exceptions with the following messages:

Error	Message
Hash algorithm not supported	Unsupported hash/integrity algorithm, hashAlgo=...
RSA encryption error	Exception when encrypting using RSA-OAEP, msg=...
AES encryption error	Exception when encrypting using AES, msg=...
Error generating hash	Exception when generating hash, msg=...
Plain text data type error	Unsupported plaintext data type: ...

B. E2EE DP C SDK

B.1 E2EE DP C SDK

Availability

The C SDK is available on Linux i86_x64/amd_64 64-bit platforms.

It is based on Open SSL version 1.0.2.



Note: Currently SSL connection to AM server was tested successfully only on Red Hat Enterprise Linux Server 7.4. Thus, SSL connection to AM server should be supported only on Red Hat Enterprise Linux Server 7.4.

Files

- **E2EEDP.h**: The header file.
- **libE2EEDPSDK.so**: The main library.
- **XMLRPC_Config_Provider.h**: The header file for the XMLRPC config provider.
- **libXMLRPC_Config_Provider.so**: The XMLRPC connectivity to AM server.

Compile time dependencies

Even though the library has been statically linked with Open SSL and XMLRPC libraries, there are some compile-time dependencies that must be resolved to compile application code with this library. These dependencies are only required at compile time; they are not required at runtime to execute compiled binaries.

1. Static gcc runtime via option "-static-libgcc"
2. Static OpenSSL **libcrypto.a**
3. Static **libxr.a**
4. GLib 2.0: **libglib-2.0.so**, **libgobject-2.0.so**, **libgio-2.0.so**
5. **libxml2.so**
6. **libdl.so**
7. **libz.so**

B.2 Using the E2EE DP C SDK

E2EEDP_Config_Provider

The default ConfigProvider for the E2EE DP C SDK is based on XML-RPC and is bundled under the **libXMLRPC_Config_Provider.so** library. It is pluggable so that a different implementation (e.g. using REST) can be injected later instead.

Getting DP Params

The main function that application calls to get DP parameters for the DP client is E2EEDP_Params_get. It returns "0" on success.

C/C++

```
typedef struct {
    int pki; // public key index
    char pkHex[1024]; // public key hex string
    char randomHex[128]; // random number hex string
    char browserCipherParams[128]; // browser cipher param
    string
    char e2eeSesToken[256]; // E2EE session ID (not used)
    char encryptedCookies[512]; // reserved
} E2EEDP_Params;

int E2EEDP_Params_get(const char *moduleId,
E2EEDP_Params *params);
```

Decrypting Data Encrypted by DP Client

To decrypt data from the DP Client, call the function E2EEDP_decrypt.

- It returns "0" on success.
- Output is a pointer to an E2EEDP_Buffer such that:
 - The buffer field must be pre-allocated with sufficient length to hold the decrypted data.
 - The length must be initially set to the length of the buffer; E2EEDP_decrypt will set it to the actual length of the decrypted data.
 - ciphertextHex : The string produced by the DP client.
 - pki : The public key index used.
 - randomNumberHex : The random number hex string used.

C/C++

```
typedef struct {
    unsigned char *buffer;
    int length;
} E2EEDP_Buffer;

int E2EEDP_decrypt(E2EEDP_Buffer *output,
    const char *moduleId,
```

```
const char* ciphertextHex,  
int pki,  
const char * randomNumberHex);
```

Initialization and Shutdown

Before using, the SDK needs to be initialized by calling E2EEDP_create.

- Returns "0" on success.
- A pointer to a E2EEDP_Config_Provider needs to be obtained first.
- providerContext is a context structure that depends on the config provider and needs to be set appropriately for that provider; in particular, the context of the XMLRPC_Config_Provider is defined as:

C/C++

```
typedef struct {  
    const char *url;  
    const int connTimeoutSecs;  
    // for SSL connection  
    const char *serverTrustStorePemFile;  
    // for SSL client authentication  
    const char *clientKeyStorePemFile;  
} XMLRPC_Context;
```

- url : The REST URL on AM server (may be http or https).
- serverTrustStorePemFile : For https, this is the full path of the PEM file that contains the trusted server certificate.
- clientKeyStorePemFile : For https with client authentication, this is the full path of the PEM file that contains the agent certificate and key pair.
- moduleIds is an array of DP module IDs.
- numModuleIds is the number of module IDs passed in.
- When application shuts down, the SDK needs to be cleaned up by calling E2EEDP_destroy.

C/C++

```
int E2EEDP_create(E2EEDP_Config_Provider *provider,  
void *providerContext,  
const char *moduleIds[], int numModuleIds);
```

```

void E2EEDP_destroy();

/***** example code of using the
XMLRPC_Config_Provider *****/
int ret;
char *moduleIds[] = {"DP001", "DP002"};
XMLRPC_Context xmlctx = {"https://am.i-
sprint.com:8443", 30,
                        "/usr/E2EEDP/truststore.pem",
                        "/usr/E2EEDP/keystore.pem"};
E2EEDP_Config_Provider *provider =
XMLRPC_getConfigProvider();

rv = E2EEDP_create(provider, &xmlctx, moduleIds, 2);
if (rv != 0) {
    // handle error
}

```

B.2 Error Codes

Error codes, as defined in **E2EEDP.h**:

C/C++

```

#define E2EEDP_Code_OutOfMemory 0x1
#define E2EEDP_Code_TypeInvalid 0x2
#define E2EEDP_Code_KeySecretInvalid 0x3
#define E2EEDP_Code_KeyPublicInvalid 0x4
#define E2EEDP_Code_KeyPrivateInvalid 0x5
#define E2EEDP_Code_KeyRSAInvalid 0x6
#define E2EEDP_Code_ModuleNotFound 0x7
#define E2EEDP_Code_RNG_Error 0x8
#define E2EEDP_Code_ConfigNotSupported 0x9
#define E2EEDP_Code_KeyNotFound 0xa
#define E2EEDP_Code_CipherTextInvalid 0xb
#define E2EEDP_Code_OutputLengthInsufficient 0xc
#define E2EEDP_Code_RSADecryptionFailed 0xd
#define E2EEDP_Code_RSAPaddingError 0xe
#define E2EEDP_Code_IntegrityLengthInsufficient 0xf
#define E2EEDP_Code_RandomNumberMismatch 0x10
#define E2EEDP_Code_DigestAlgoNotSupported 0x11
#define E2EEDP_Code_DigestLengthInvalid 0x12
#define E2EEDP_Code_DigestMismatch 0x13
#define E2EEDP_Code_SymmetricAlgoNotSupported 0x14
#define E2EEDP_Code_SymmetricCipherError 0x15
#define E2EEDP_Code_DigestError 0x16

```
